

Aproximación de "curvas"

Interpolación → Splines → Bézier → B-splines → LS → PCA

Manuela Bastidas Olivares

Una sola pregunta conduce toda la clase

¿Cómo representar datos con funciones suaves?

Todos los métodos son: $f(t) = \sum_i c_i \phi_i(t)$

La diferencia está en la **elección de la base** $\{\phi_i\}$



1 Interpolación

Retomemos: bases, sistemas y el problema de Runge

Todo es una base: monomial, Lagrange, Hermite

$$A \mathbf{c} = \mathbf{y}, \quad A_{ki} = \phi_i(t_k), \quad \text{sistema cuadrado } (n+1) \times (n+1)$$

Monomial $\phi_i = t^i$

A = Vandermonde

Mal condicionada para n
grande

```
# numpy lo hace  
directo  
c = np.polyfit(t,y,  
              n)  
p = np.polyval(c,  
              t_n)
```

Lagrange $\phi_i(t_j) = \delta_{ij}$

$A = I$ trivial: $c_i = y_i$

Oscila en extremos (Runge)

```
from scipy.  
    interpolate \  
import lagrange  
p = lagrange(t,y)  
y_new = p(t_n)
```

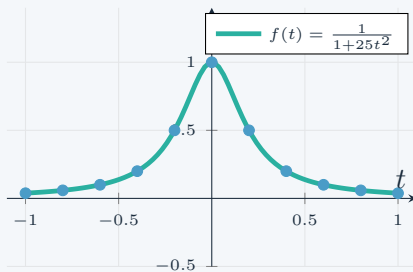
Hermite valores+derivadas

$2(n+1)$ condiciones

Controla tangentes

```
from scipy.  
    interpolate \  
import  
    CubicHermiteSpline
```

El problema de Runge: por qué la base importa



La base monomial/Lagrange es **global**: cada ϕ_i influye en todo $[-1, 1]$.

Runge

Grado n alto + nodos equiespaciados
= oscilaciones explosivas en los
bordes.

La solución

Bases con **soporte local**.
La oscilación no se propaga

Splines cúbicos: base local por tramos

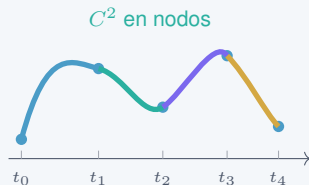
En cada $[t_k, t_{k+1}]$:

$$S_k(t) = a_k + b_k h + c_k h^2 + d_k h^3, \quad h = t - t_k.$$

Condiciones: S, S', S'' continuas en nodos.

$$A \mathbf{c} = \mathbf{y}, \quad A \text{ tridiagonal}$$

```
from scipy.interpolate \
import CubicSpline
cs = CubicSpline(t, y)
# base local automatica
y_new = cs(t_new)
y_der = cs(t_new, 1) # S'
```



La base son las **hat functions** cúbicas — soporte local

2

Bézier y B-splines

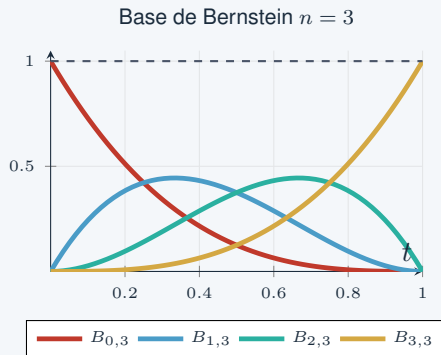
La base como herramienta de diseño

Base de Bernstein: bella pero global

$$B_{i,n}(t) = \binom{n}{i} t^i (1-t)^{n-i}, \quad \phi_i = B_{i,n}$$

La curva de Bézier es:

$$\mathbf{C}(t) = \sum_{i=0}^n B_{i,n}(t) \mathbf{P}_i$$

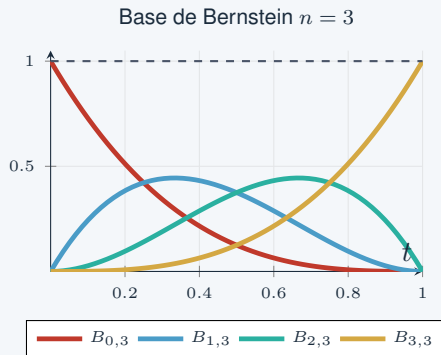


Mover $\mathbf{P}_i$ **mueve toda la curva**
porque $B_{i,n}$ es no nula en $[0, 1]$

Base de Bernstein: bella pero global

- $\sum_i B_{i,n}(t) = 1$ (partición)
- $B_{i,n} \geq 0$ (no negativa)
- $C(0) = P_0, C(1) = P_n$
- Soporte $[0, 1]$ completo

```
from scipy.special import  
    comb  
def bernstein(i, n, t):  
    return comb(n,i)*t**i*(1-  
        t)**(n-i)  
# curva: suma ponderada  
C = sum(bernstein(i,n,t)*  
        P[i]  
        for i in range(n+1))
```



Mover P_i mueve toda la curva
porque $B_{i,n}$ es no nula en $[0, 1]$

Comparación directa de bases

Base	Soporte	Sistema	Interpola	Código
Monomial t^i	global	Vandermonde	sí	<code>np.polyfit</code>
Lagrange	global	$A = I$	sí	<code>lagrange(t,y)</code>
Spline cúbico	local	tridiagonal	sí	<code>CubicSpline</code>
Bernstein $B_{i,n}$	global	no-interpola	sí extremos	manual
B-spline $N_{i,p}$	local	no-interpola	sí extremos	<code>BSpline</code>

Todos son $f(t) = \sum c_i \phi_i(t)$. La base define la estructura de A y las propiedades de f .

3

Mínimos cuadrados

Cuando los datos superan a los parámetros

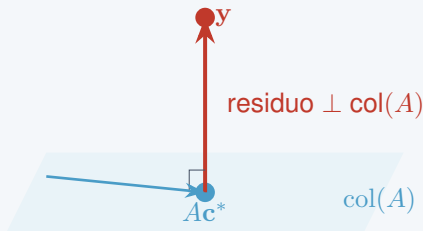
Sistema sobredeterminado: proyección ortogonal

N datos, $n + 1$ funciones base,
 $N \gg n + 1$.

El sistema $A\mathbf{c} = \mathbf{y}$ ($A \in \mathbb{R}^{N \times (n+1)}$) no tiene solución exacta.

$$\min_{\mathbf{c}} \|A\mathbf{c} - \mathbf{y}\|_2^2$$

$$A^T A \mathbf{c}^* = A^T \mathbf{y} \quad (\text{ecuaciones normales})$$



```
import numpy as np
# A: matriz de colocacion
# y: datos
c, res, rank, sv = np.linalg.
    lstsq(
        A, y, rcond=None)
# equivalente: resolver A^T A c
    = A^T y
```

4 PCA

La base óptima emerge de los datos

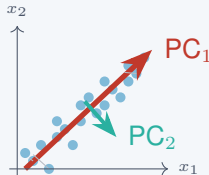
PCA: el mismo problema, base adaptativa

Datos: $X \in \mathbb{R}^{N \times d}$ (N observaciones, d variables).

Mismo problema que LS: aproximar X con un subespacio de dimensión $k < d$.

$$\min_{\text{rango}(B) \leq k} \|X - B\|_F^2$$

Diferencia clave: la **base** $\{\phi_i\}$ no es fija, se optimiza.



```
from sklearn.decomposition
import PCA
pca = PCA(n_components=2)
pca.fit(X)
```

Teorema de Eckart–Young: PCA es LS en matrices

Teorema de Eckart–Young (1936)

La mejor aproximación de rango k de $\tilde{X}$ en norma de Frobenius es:

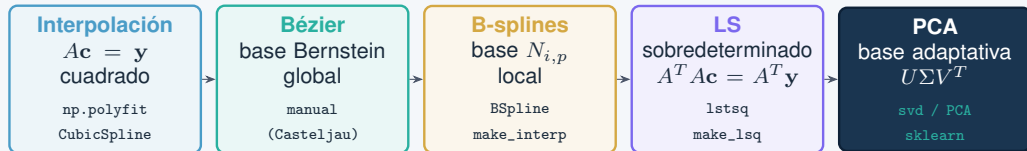
$$B = U_k \Sigma_k V_k^T = \sum_{i=1}^k \sigma_i \mathbf{u}_i \mathbf{v}_i^T$$

donde U_k, Σ_k, V_k son las primeras k columnas/valores singulares de la SVD.

Es el mismo problema que LS, pero en espacio de matrices:

$$\min_{\text{rango}(B) \leq k} \left\| \tilde{X} - B \right\|_F^2 = \sum_{i=k+1}^d \sigma_i^2$$

Una sola idea



Todos los métodos son $f = \sum_i c_i \phi_i$.

Lo que cambia es cómo **eliges la base**